

Guide 2

The FUSE Framework

2026

The main goal of this guide is to understand how to use the FUSE framework to implement custom File Systems (FSs) in user space. We will use as an example an adapted version of the pass-through FUSE implementation available at <https://github.com/libfuse/libfuse>. Figure 1 overviews how the framework and the pass-through implementation work. Also, consider reading <https://www.usenix.org/system/files/conference/fast17/fast17-vangoor.pdf> for further details about the framework.

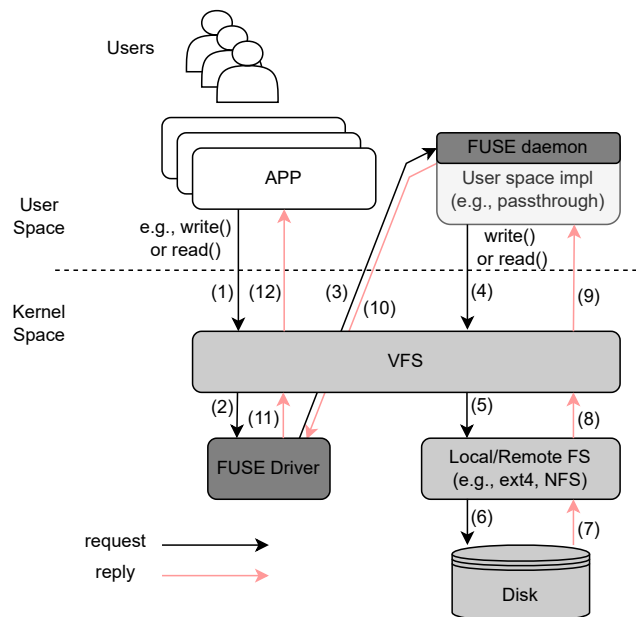


Figure 1: Simplified FUSE Framework architecture and flow of events for a pass-through implementation

1 Setup

Use the VM from *Guide0* for the next exercises. Also, we must prepare the software and OS environment for this guide.

1. Install the FUSE library.

```
sudo apt install libfuse3-dev
```

2. Copy the source code provided along with the guide to the VM.
3. Create two directories (e.g., `/mnt/fs` and `/backend`).

2 FUSE Pass-through

Now, let us compile and test the FUSE pass-through implementation.

1. Compile the code.

```
gcc -Wall passthrough.c `pkg-config fuse3 --cflags --libs` -o passthrough
```

2. Mount the FUSE file system.

Arguments: `/mnt/fs` specifies the root folder exposed by the pass-through FS for users.

The `subdir` module defines the path of the backend (i.e., `/backend`) where the FUSE FS will actually store and access data from.

The `allow_other` option allows all users to access the FS.

The `-f` flag runs the program in foreground mode.

```
sudo ./passthrough /mnt/fs -omodels="subdir,subdir=/backend" -oallow_other -f
```

3. Experiment creating, editing and removing files at `/mnt/fs`. While doing it, take a look at the `/backend` folder.

3 Exercises

1. Try creating a folder at the FS. The operation fails because there is no support for creating directories at the current implementation. Inspect the `passthrough.c` code and update it to support creating and removing directories (i.e., `mkdir` and `rmdir`).

Hint1: Look at the `fuse_operations` struct to know more details about each operation supported by the pass-through implementation (<https://libfuse.github.io/doxygen/annotated.html>)

2. Our current implementation already contains code to instrument and debug `open/create` operations handled by the FS. Expand this debug code to also include `read`, `write`, and `close` operations.

Hint1: Take a look at the `fuse_context` struct and `fuse_get_context()` function (<https://libfuse.github.io/doxygen/annotated.html>) to understand how debugging information is being collected and how the `struct context` is being passed across functions without using global variables.

3. Update the FS code to apply data transformations before storing the data in the `/backend` folder. As an example, let's assume that we want to obfuscate the file's data stored at the backend. For that, we increment by a given *value* the bytes of each file before being written. Then, when reading the bytes, we want users to observe non-obfuscated content.

Hint1: Use the `add` and `sub` auxiliary functions provided in the pass-through helper code.

Note: This is a simple and unrealistic example to better understand how our FS could provide other useful data transformations such as transparent data encryption, compression, ...

4 Extras / Homework

1. Our debug operation counters are not thread-safe. In fact, FUSE can use multiple threads to serve application requests. Make the code thread-safe (without using global variables).
2. Instead of writing debug information to the *stdout*, redirect it to a log file (e.g., */tmp/debug.log*). You can use `fopen`, `fprintf`, and related calls for this purpose.

Learning outcomes

Experiment the FUSE framework. Assess how data transformations can be implemented in user space file systems. Experiment with file system debugging.